Alta disponibilidad en el acceso a una base de datos

Informe técnico (borrador)

Curso	[Nombre del curso]
Catedrático	[Nombre]
Integrantes	[Nombre · carné], [Nombre · carné], [Nombre · carné]
Fecha	[Fecha de entrega]
Repositorio	github.com/NohelEstradaP/ha_db_ip

> Los campos entre corchetes se completan antes de entregar. Los resultados de §7 son mediciones reales en las 3 máquinas del equipo (23 de octubre de 2026). Las pruebas marcadas como *pendiente* deben ejecutarse físicamente en n2 o n3.

1. Introducción

Una base de datos instalada en un solo equipo es un **punto único de falla**: si ese equipo se apaga, se cuelga o pierde la red, todas las aplicaciones que dependen de ella se detienen y los datos recientes pueden perderse. Este proyecto implementa y prueba una solución de **alta disponibilidad** con tres equipos físicos. Cada uno mantiene una copia completa de la base de datos, y el sistema sigue funcionando sin intervención humana cuando cualquiera de ellos se desconecta.

2. Objetivos

2.1 Requerimientos del enunciado

#	Requerimiento
R1	La base de datos recibe al menos **90 cambios por minuto**.
R2	Se accede desde **dos o más equipos** y siempre existe algún **mirror** (copia).
R3	En un momento dado **uno de los equipos se apagará**.
R4	La **integridad y funcionalidad continúan** mientras uno de los equipos está desconectado.
R5	Al **restablecer la comunicación**, el equipo se **vuelve a sincronizar**.

2.2 Objetivos específicos

1. Desplegar un clúster PostgreSQL de 3 nodos en 3 máquinas físicas con sistemas operativos y arquitecturas de CPU distintos.
2. Lograr failover automático, sin intervención manual, cuando falla el nodo primario.
3. Garantizar que no se pierdan ni se dupliquen transacciones confirmadas (RPO = 0).
4. Medir el tiempo de recuperación (RTO) en distintos escenarios de falla.
5. Demostrar la re-sincronización automática del nodo que regresa.

3. Marco teórico

Replicación física por streaming. PostgreSQL registra cada cambio en el *Write-Ahead Log* (WAL) antes de aplicarlo a los archivos de datos. Una réplica recibe ese WAL en tiempo real desde el primario y lo reproduce, de modo que mantiene una copia idéntica byte a byte que además se puede consultar en modo solo lectura (*hot standby*).

Replicación síncrona vs. asíncrona. En modo asíncrono, el primario confirma el `COMMIT` sin esperar a la réplica; si cae en ese instante, la transacción puede perderse. En modo síncrono, el `COMMIT` solo se confirma cuando al menos una réplica ya escribió el WAL en disco. Así, toda transacción confirmada existe en al menos dos equipos.

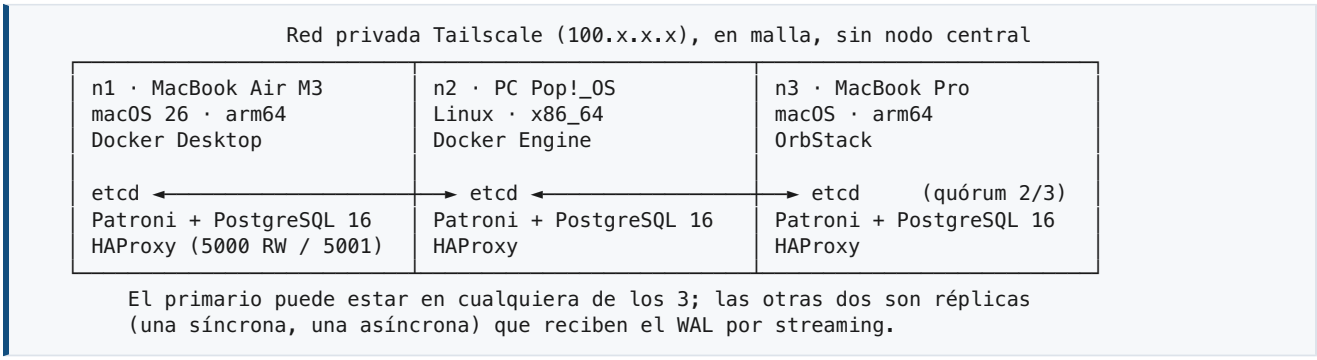
RPO y RTO. El *Recovery Point Objective* es cuántos datos se pueden perder ante una falla (aquí, 0 transacciones confirmadas). El *Recovery Time Objective* es cuánto tarda el servicio en volver a aceptar escrituras.

Consenso y quórum (Raft / etcd). Para decidir qué nodo es el primario sin ambigüedad se usa etcd, un almacén distribuido basado en el algoritmo Raft. Un cambio solo se acepta si lo confirma la **mayoría** de los miembros: con 3 nodos, 2. Con N nodos se toleran $\lfloor (N-1)/2 \rfloor$ fallas, es decir, 1 con 3 nodos.

Split-brain. Si dos nodos creen al mismo tiempo que son el primario y ambos aceptan escrituras, los datos divergen y no se pueden reconciliar. Con 2 nodos no es posible distinguir "el otro murió" de "se cortó la red". Con 3 nodos y quórum, el lado en minoría sabe que está aislado y deja de aceptar escrituras.

Teorema CAP. Ante una partición de red, un sistema distribuido debe elegir entre consistencia y disponibilidad. Este diseño elige **consistencia**: el lado en minoría deja de escribir, y el lado con mayoría sigue disponible.

4. Arquitectura



Componente	Versión	Función
PostgreSQL	16.15	Motor de base de datos
Patroni	4.0.4	Gestiona cada PostgreSQL: replicación, failover automático, `pg_rewind`, API REST de salud
etcd	3.5.17	Consenso (Raft): guarda quién es el líder y exige mayoría
HAProxy	3.0	Puerto 5000 siempre hacia el primario; 5001 hacia las réplicas
Tailscale	—	VPN en malla: IP fija por equipo, misma red o redes distintas
Docker	28/29 · OrbStack	Misma imagen (Debian 12, misma glibc) en macOS y Linux

4.1 Decisiones de diseño

1. **Tres nodos.** Es el mínimo que permite quórum y evita el split-brain.
2. **Replicación síncrona** (`synchronous_mode: true`): RPO = 0. Patroni solo puede promover a la réplica síncrona, que tiene todo lo confirmado.
3. **Docker en los 3 equipos.** La replicación física copia archivos byte a byte; usar la misma imagen Debian en macOS y Linux elimina diferencias de colación (`initdb --locale=C.UTF-8`) y de versiones.
4. **`pg_rewind` y *data checksums*.** El primario caído vuelve como réplica sin copiar toda la base.
5. **Tiempos de Patroni** `ttl=20 s`, `loop_wait=5 s`, `retry_timeout=5 s`: failover en unos 20–30 s ante una caída brusca.
6. **Acceso de la aplicación** por cadena libpq con los 3 hosts y `target_session_attrs=read-write`. El cliente encuentra solo al primario, sin depender de una IP virtual.

5. Implementación

La configuración completa está en el repositorio. Cada equipo se prepara con `./instalar.sh`, que instala Docker o detecta OrbStack, instala Tailscale, Python y las imágenes, y genera el archivo `.env` con la identidad del nodo, las IP de los 3 y las contraseñas compartidas. Luego:

1. `./scripts/iniciar.sh` en los 3 equipos: levanta etcd, Patroni/PostgreSQL y HAProxy, espera quórum y primario.
2. `./scripts/crear_bd.sh` una vez: crea el usuario `app`, la base `hadb` y el esquema.
3. `./scripts/panel.sh`: panel web en tiempo real (`http://<IP>:8080`).

Configuración clave de Patroni (`patroni/patroni.yml`):

```
bootstrap:
  dcs:
    ttl: 20
    loop_wait: 5
    retry_timeout: 5
    maximum_lag_on_failover: 1048576
    synchronous_mode: true
    synchronous_node_count: 1
    postgresql:
      use_pg_rewind: true
      use_slots: true
      parameters: { wal_level: replica, wal_log_hints: "on", wal_keep_size: 1GB }
  initdb: [ { encoding: UTF8 }, { locale: C.UTF-8 }, data-checksums ]
```

6. Diseño de la prueba de carga e integridad

Modelo de datos. 100 cuentas con Q10,000.00 cada una. Cada operación del generador es una **transferencia**: 1 `INSERT` en `movimientos` y 2 `UPDATE` en `cuentas`, es decir, 3 cambios por transacción. Un generador a 1 transacción por segundo produce **180 cambios por minuto**, el doble del mínimo. Con un generador por equipo se llega a 540.

Invariantes que prueban la integridad:

1. `SUM(saldo) = 1,000,000.00` siempre: las transferencias mueven dinero pero no lo crean ni lo destruyen. Una transacción aplicada a medias o duplicada cambiaría la suma.
2. Cada generador numera sus transacciones `seq = 1, 2, 3…` con la restricción `UNIQUE (generador, seq)`, así que `MAX(seq) = COUNT(*)` significa que no falta ninguna.
3. Con la carga detenida, el hash md5 de ambas tablas es idéntico en los 3 nodos.

Idempotencia. Si la conexión se cae durante el `COMMIT`, el cliente no sabe si la transacción se guardó. El generador reintenta la **misma** `seq`, y `ON CONFLICT DO NOTHING` evita el duplicado. Esto da semántica de "exactamente una vez".

Herramientas: - `generador.py`: carga, con reintentos y recuperación del atraso. - `verificar.py`: compara los invariantes y hashes en los 3 nodos. - `panel.py`: estado en vivo, eventos y duración de cada failover.

7. Resultados

7.1 Validación previa en laboratorio

Antes de usar las 3 máquinas se probó el mismo diseño con los 3 nodos en un solo equipo (`laboratorio/`): failover por apagado de 22–26 s, partición de 27 s, reingreso con `pg_rewind` y verificador OK en todas las pruebas.

7.2 Compatibilidad entre arquitecturas (ARM ↔ x86)

El clúster arrancó con **n3 (Mac, arm64)** como primario y **n2 (PC, x86_64)** copió y replicó desde él. Luego el primario pasó n3 → n1 → n2, y con **n2 (x86)** como primario las dos Mac replicaron con los datos intactos (100 cuentas, suma 1,000,000). La replicación física funciona en **ambas direcciones** entre arquitecturas, porque los 3 nodos usan la misma imagen de PostgreSQL 16.15 sobre Debian 12.

7.3 Pruebas de falla en las máquinas reales

Condiciones: los 3 nodos sanos con un generador a 180 cambios por minuto antes de cada prueba. Las fallas de n1 se provocaron con `docker kill` de sus 3 contenedores, que desde el clúster equivale a cortar la corriente del nodo, y con `tailscale down` para la partición.

Prueba	Escenario	Nodo afectado (rol)	Escrituras en pausa (RTO)	Suma OK	Faltan	Re-sincronización
P8	Switchover planificado	n2 (primario) → n1	**3,0 s**	✓	0	3 nodos sanos en 14 s
P9	Detención limpia de Patroni en el primario	n1 (primario)	**3,0 s** (nuevo primario n3)	✓	0	n1 réplica en 3 s
P3	**Caída brusca del primario**	n1 (primario)	**26,1 s** (nuevo primario n3)	✓	0	`pg_rewind` desde n3; sano en 45 s
P2	Caída brusca de una réplica	n1 (réplica)	**0 s** (sin failover)	✓	0	streaming en 4 s
P4	**Partición de red del primario**	n1 (primario)	n1 se degradó solo a los **11 s**; n3 promovido a los **22 s**	✓	0	réplica en timeline 11, lag 0
P7	Desconexión prolongada (10 min) con carga	n1 (réplica)	0 s	✓	0	600 transacciones atrasadas recuperadas por WAL en **5 s**
P1	Apagado normal de una réplica	n2 o n3	*pendiente*			
P5	Réplica sin red	n2 o n3	*pendiente*			
P6	Reinicio completo del equipo	n2 o n3	*pendiente*			

Durante P3, con n1 caído 2 minutos, el clúster se mantuvo en estado "Degradado: 2 de 3 nodos" con **180 cambios por minuto**, suma 1,000,000.00 y 0 secuencias faltantes.

7.4 Evidencia del comportamiento ante partición (P4)

Registro de Patroni en n1, el primario aislado:

```
01:30:22 ERROR: Error communicating with DCS
01:30:22 INFO: demoting self because DCS is not accessible and I was a leader
01:30:22 INFO: Demoting self (offline)
```

Historial del clúster (`patronictl history`): el timeline 10 se abrió en **n3** a las 01:30:33 UTC. En ningún momento hubo dos primarios aceptando escrituras.

7.5 Verificación final de integridad

Con la carga detenida, después de todas las pruebas:

```
nodo  rol      cuentas           suma     movs faltan  md5 cuentas  md5 movs
n1    réplica      100   1,000,000.00     1301      0  7a6e1a907591 ea098493b542
n2    réplica      100   1,000,000.00     1301      0  7a6e1a907591 ea098493b542
n3    PRIMARIO     100   1,000,000.00     1301      0  7a6e1a907591 ea098493b542

RESULTADO: OK. 3 nodos con datos idénticos, suma = 1,000,000, sin secuencias faltantes, un solo
primario.
```

Cero pérdida de transacciones confirmadas. El generador registró 1,301 transacciones confirmadas y la base tiene exactamente 1,301 movimientos, con `seq` 1 a 1,301 sin huecos ni repetidos. Hubo 8 transacciones que necesitaron un segundo intento, una por cada cambio de primario, y todas quedaron guardadas una sola vez:

Hora	seq	Latencia	Escrita finalmente en	Causa
19:22:53	74	1,7 s	n1	P8 switchover
19:23:20	102	0,8 s	n3	P9 detención limpia
19:24:14	153	3,9 s	n2	switchover de preparación
19:24:26	167	1,8 s	n1	switchover de preparación
19:25:24	198	29,1 s	n3	**P3 caída brusca del primario**
19:29:30	467	5,5 s	n2	switchover de preparación
19:29:45	486	1,8 s	n1	switchover de preparación
19:31:09	515	56,9 s	n3	**P4 partición** (el generador estaba del lado aislado)

8. Análisis

- **R1 (≥ 90 cambios/min):** se cumple con un solo generador (180/min) y con margen amplio (540/min) con uno por equipo. Durante un failover las escrituras se pausan unos segundos, pero el generador recupera el atraso y el minuto completo se mantiene por encima de 90.
- **R2 (mirror permanente):** en todo momento hubo al menos una réplica síncrona. Cada transacción confirmada existía en 2 equipos.
- **R3/R4 (un equipo se apaga y todo continúa):** en todas las pruebas el clúster siguió aceptando escrituras con los otros dos nodos, sin pérdida ni duplicación (suma constante y 0 faltantes).
- **R5 (re-sincronización):** el nodo que regresa se reincorpora solo. Si era réplica, recupera el WAL atrasado. Si era primario, `pg_rewind` descarta lo que nunca se confirmó y lo pone a seguir al nuevo primario.
- **RTO.** Ante una caída limpia o planificada es de unos 3 s, porque Patroni libera el liderazgo de inmediato. Ante una caída brusca es de unos 26 s: el `ttl` de 20 s más la detección. Se puede reducir bajando `ttl`, a costa de más riesgo de failovers falsos en redes inestables.
- **RPO = 0** gracias a la replicación síncrona: en ninguna prueba se perdió una transacción confirmada.
- **Partición:** el nodo aislado se degradó solo antes de que el otro lado promoviera un nuevo primario (11 s contra 22 s). Así se evita el split-brain.

9. Limitaciones y mejoras

- Con 3 nodos se tolera **una** falla. Con dos simultáneas, el nodo restante queda en solo lectura, y es lo correcto para proteger la integridad. Para tolerar 2 fallas se necesitarían 5 nodos.
- Sin cifrado TLS entre nodos: el tráfico va dentro de Tailscale, que ya cifra con WireGuard, pero en producción convendría TLS en PostgreSQL, etcd y la API de Patroni.
- Sin respaldos ni recuperación a un punto en el tiempo (PITR): agregar pgBackRest.
- Sin *watchdog* de hardware: en producción ayuda a garantizar que un primario colgado se detenga.
- Monitoreo: el panel propio cubre la demostración; en producción, Prometheus y Grafana con alertas.

10. Conclusiones

1. Un clúster PostgreSQL + Patroni + etcd de 3 nodos cumple todos los requerimientos. La base sigue aceptando escrituras cuando cualquier equipo se apaga o pierde la red, y el equipo que regresa se re-sincroniza sin intervención.
2. La replicación síncrona más la aplicación idempotente garantizan que no se pierdan ni se dupliquen transacciones confirmadas, lo que se comprobó con invariantes de negocio.
3. El quórum de 3 nodos evita el split-brain: el nodo aislado se degrada solo.
4. Usar contenedores con la misma imagen permitió mezclar macOS (ARM) y Linux (x86) en un mismo clúster de replicación física.

Anexos

- **A.** Plan de implementación completo: `PLAN_ALTA_DISPONIBILIDAD.md` / `.pdf`
- **B.** Guía de instalación y pruebas: `GUIA_PENDIENTES.md` / `.pdf`
- **C.** Archivos de configuración: `docker-compose.yml`, `patroni/`, `haproxy/`, `sql/esquema.sql`
- **D.** Programas: `app/generador.py`, `app/verificar.py`, `app/monitor.py`, `app/panel.py`
- **E.** Evidencias: carpeta `evidencias/` (logs de Patroni, CSV de los generadores y del monitor, capturas del panel)